BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 4

Roll no: 51 PRN No: 12411033 Name of Student: Manjiri Kulkarni

Subject Name: Deep Learning

Problem Statement:

Develop an LSTM based model for time-series forecasting using stock price, weather or sales datasets.

Algorithm:

1. Start the program.
2. Import the required libraries such as TensorFlow, Keras, NumPy, Pandas, Matplotlib, and Scikit-learn.
3. Load the Gold Price dataset using KaggleHub.
4. Preprocess the data by selecting Date and INR columns, handling missing values, and sorting by date.
5. Visualize historical gold prices using a line graph.
6. Normalize gold prices using MinMaxScaler.
7. Create input sequences using the previous 30 days of gold prices.
8. Split the data into training and testing sets in an 80:20 ratio.
9. Build and compile an LSTM model using 50 LSTM units, a Dense output layer, the Adam optimizer, and Mean Squared Error loss.
10. Train the model for 10 epochs and predict gold prices on the testing dataset.
11. Calculate MAE and RMSE, and plot actual versus predicted prices and training versus validation loss.
12. Stop the program.

Code:

```
import kagglehub
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
path = kagglehub.dataset_download(
    "rizkykiky/gold-price-dataset"
)
print("Path to dataset files:", path)
import os
print("Files in dataset folder:")
print(os.listdir(path))
df = pd.read_csv(os.path.join(path, "Daily.csv"))
print(df.head())
print(df.columns)
df["Date"] = pd.to_datetime(df["Date"])
df = df[["Date", "INR"]]
df = df.dropna()
df = df.sort_values("Date")
print(df.head())
print(df.shape)
plt.figure(figsize=(12, 5))
plt.plot(df["Date"], df["INR"])
plt.xlabel("Date")
plt.ylabel("Gold Price (INR)")
plt.title("Gold Price in INR")
plt.show()
from sklearn.preprocessing import MinMaxScaler
# Remove commas from INR values and convert them to numbers
df["INR"] = df["INR"].str.replace(",", "", regex=False)
df["INR"] = pd.to_numeric(df["INR"], errors="coerce")
# Remove any values that could not be converted
df = df.dropna(subset=["INR"])
prices = df["INR"].values.reshape(-1, 1)
scaler = MinMaxScaler()
prices_scaled = scaler.fit_transform(prices)
import numpy as np
sequence_length = 30
X = []
y = []

for i in range(sequence_length, len(prices_scaled)):
    X.append(prices_scaled[i-sequence_length:i])
```

```
    y.append(prices_scaled[i])
X = np.array(X)
y = np.array(y)
print("X shape:", X.shape)
print("y shape:", y.shape)
train_size = int(len(X) * 0.8)
X_train = X[:train_size]
X_test = X[train_size:]
y_train = y[:train_size]
y_test = y[train_size:]
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
model = Sequential()
model.add(
    LSTM(
        50,
        input_shape=(X_train.shape[1], X_train.shape[2])
    )
)
model.add(Dense(1))
model.compile(
    optimizer="adam",
    loss="mean_squared_error"
)
model.summary()
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_data=(X_test, y_test)
)
predictions = model.predict(X_test)
predicted_prices = scaler.inverse_transform(predictions)
actual_prices = scaler.inverse_transform(y_test)
print("Predicted prices:")
print(predicted_prices[:5])
print("Actual prices:")
print(actual_prices[:5])
from sklearn.metrics import mean_absolute_error, mean_squared_error
mae = mean_absolute_error(actual_prices, predicted_prices)
rmse = np.sqrt(
    mean_squared_error(actual_prices, predicted_prices)
)
```

```
print("MAE:", mae)
print("RMSE:", rmse)
plt.figure(figsize=(12, 5))
plt.plot(actual_prices, label="Actual Price")
plt.plot(predicted_prices, label="Predicted Price")
plt.xlabel("Time")
plt.ylabel("Gold Price (INR)")
plt.title("Actual vs Predicted Gold Price")
plt.legend()
plt.show()
plt.figure(figsize=(10, 5))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.show()
```

Output:



